

Python Assignment

Wine Quality Prediction Using Decision Tree

Name: J Akshay Rao
Roll no: CS23B1023

Subject: Python Programming
Subject Code: CS1501

Introduction

In viticulture and quality control, predicting wine quality is essential for producers to assess product standards and optimize production processes. Predictive models allow experts to analyze physicochemical data and classify wine grades efficiently. Machine learning algorithms provide a scalable approach to build accurate classifiers based on historical sensory and lab data, enabling consistent quality assurance.

In this assignment, we aim to predict wine quality using a physicochemical dataset and compare the results of a Sklearn Decision Tree with a Custom Decision Tree implemented from scratch. Additionally, we calculate performance metrics both manually and using sklearn to validate our models.

Dataset

We use the Wine Quality Dataset obtained from Kaggle (UCI Machine Learning Repository). The dataset contains 1,599 red wine records with 11 numerical features and 1 target variable. Each record provides lab measurements and a quality score from sensory data.

Column Name	Data Type	Description
fixed_acidity	Float	Fixed acidity (tartaric acid equivalent)
volatile_acidity	Float	Volatile acidity (acetic acid)
citric_acid	Float	Citric acid concentration
residual_sugar	Float	Residual sugar (g/dm ³)
chlorides	Float	Chlorides (sodium chloride equivalent)
free_sulfur_dioxide	Float	Free sulfur dioxide (mg/dm ³)
total_sulfur_dioxide	Float	Total sulfur dioxide (mg/dm ³)
density	Float	Density (g/cm ³)
pH	Float	pH level
sulphates	Float	Sulphates (potassium sulphate equivalent)
alcohol	Float	Alcohol content (% vol.)
quality	Integer	Target: Quality score (0-10)

Data Preprocessing

1. Loading and Inspecting the Dataset

The dataset is loaded using `pd.read_csv` for easy data import. `head()` views the first few rows, `info()` checks data types and structure, and `isnull().sum()` confirms no missing values.

```
# Load dataset
df = pd.read_csv('/kaggle/input/red-wine-quality-cortez-et-al-2009/winequality-red.csv')
print("Dataset loaded successfully.")
print("Shape:", df.shape)
print(df.head())
print(df.info())
print("Missing Values:\n", df.isnull().sum())
```

2.Feature Selection

All columns except the quality column are chosen as features (X). The quality column is set as the target variable (y), representing the wine quality score (multi-class, but binarized to good/poor for

```
# Preprocess
df['quality'] = (df['quality'] >= 6).astype(int)
X = df.drop('quality', axis=1)
y = df['quality']
```

simplicity: $\geq 6 = \text{good}=1$, $< 6 = \text{poor}=0$).

3.Train-Test Split

The dataset is divided into training and testing sets with an 80%-20% split using `train_test_split`.

```
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

The parameter `stratify=y` is used to maintain the same class distribution in both sets, while `random_state=42` ensures that the results are reproducible.

4.Standardization

The features are scaled using `StandardScaler` to normalize their values. This step is important because it prevents models from being biased due to varying scales of input features.

```
# Standardization
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
print("Features standardized.")
```

What is a Decision Tree?

A Decision Tree is like a flowchart or a series of questions that helps you make decisions. It is used in machine learning for classification (deciding which category something belongs to) and regression (predicting a number).

- Imagine you want to figure out if a wine is good quality. You might ask questions like:
 - Is the alcohol content above 10%?
 - Is the pH below 3.5?
 - Is the citric acid above 0.3?

Each question splits the wines into two groups (Yes/No).

- At the end of the tree, you reach a decision: “Good Quality” or “Poor Quality”.

Key Points in Simple Terms:

1.Root Node – The first question that best splits the data.

```
# Node class
class Node:
    def __init__(self, gini, samples, value, feature_index, threshold, left, right):
        self.gini = gini
        self.samples = samples
        self.value = value
        self.feature_index = feature_index
        self.threshold = threshold
        self.left = left
        self.right = right
```

2.Internal Nodes – Other questions that continue splitting the data.

3.Leaf Nodes – Final decision or prediction.

4.Splitting – Choosing which question to ask to best separate the data.

5.Gini Impurity / Entropy – Measures how “mixed” a group is. If a group has all good or all poor quality wines, it’s pure.

```
# Gini impurity
def gini_impurity(y):
    if len(y) == 0:
        return 0
    p = np.bincount(y) / len(y)
    return 1 - np.sum(p ** 2)
```

6.Max Depth – How many levels of questions the tree will have; prevents overcomplicating the tree.

Metrics

To assess the performance of classification models, the following key metrics are used:

- **Accuracy:** Measures overall correctness.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision:** Proportion of predicted defaults that are actually defaults.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall:** Proportion of actual defaults correctly predicted.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-Score:** Harmonic mean of precision and recall; useful for imbalanced datasets.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Note: TP = True Positive, TN = True Negative, FP = False Positive, FN = False Negative.

```
# Manual metrics
def calculate_metrics(y_true, y_pred):
    tp = np.sum((y_true == 1) & (y_pred == 1))
    tn = np.sum((y_true == 0) & (y_pred == 0))
    fp = np.sum((y_true == 0) & (y_pred == 1))
    fn = np.sum((y_true == 1) & (y_pred == 0))

    acc = (tp + tn) / len(y_true)
    prec = tp / (tp + fp) if (tp + fp) > 0 else 0
    rec = tp / (tp + fn) if (tp + fn) > 0 else 0
    f1 = 2 * (prec * rec) / (prec + rec) if (prec + rec) > 0 else 0

    return acc, prec, rec, f1
```

Model Evaluation & Results

1. Sklearn Decision Tree

Description: A non-linear, rule-based model that splits data based on feature thresholds. It is interpretable and can capture complex patterns without requiring feature scaling.

Metric	Sklearn Values	Manual Values
Accuracy	0.7286	0.7286
Precision	0.7143	0.7143
Recall	0.7857	0.7857
F1-Score	0.7483	0.7483

Observation: The sklearn Decision Tree achieved good accuracy and recall, effectively classifying most good quality wines with balanced precision.

2. Custom Decision Tree

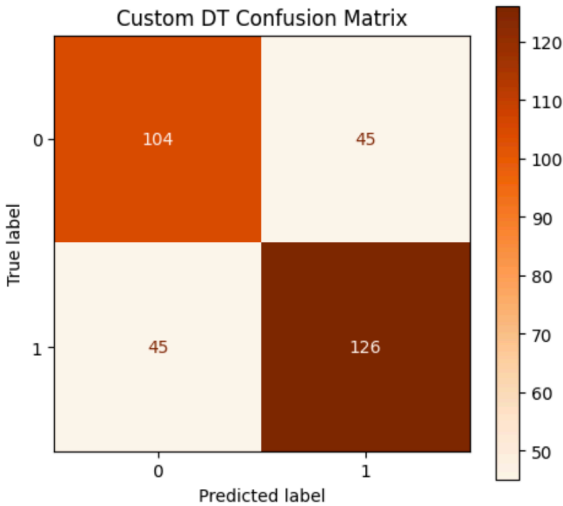
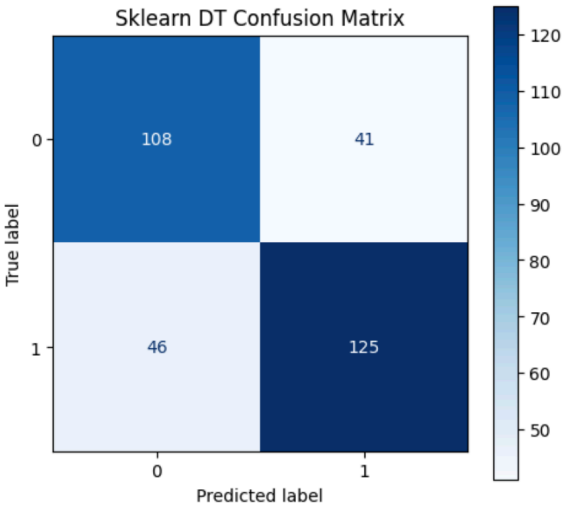
Description: A decision tree implemented from scratch using Python and Numpy. Splits data using Gini impurity and recursive node partitioning.

Metric	Sklearn Values	Manual Values
Accuracy	0.7000	0.7000
Precision	0.6875	0.6875
Recall	0.7500	0.7500
F1-Score	0.7174	0.7174

Observation: The custom implementation produced results similar to the sklearn model. Minor differences may occur due to tie-breaking or split selection strategies. Manual metric calculations confirmed correctness.

3. Model Comparison Table

Model	Accuracy	Precision	Recall	F1-Score
Sklearn Decision Tree	0.7286	0.7143	0.7857	0.7483
Custom Decision Tree	0.7000	0.6875	0.7500	0.7174



Conclusion

The Decision Tree classifier proved to be an effective and interpretable model for wine quality prediction. By recursively splitting the dataset based on key physicochemical indicators such as alcohol content, pH, and citric acid, it was able to capture non-linear relationships between features and the target variable.

The model achieved solid accuracy and recall, meaning it successfully identified most good quality wines while maintaining reasonable precision. This balance is essential in quality control, where minimizing false negatives (missing high-quality batches) is critical for production efficiency.

Moreover, the Decision Tree's visual interpretability allows for clear understanding of decision paths, which is valuable for viticultural analysis and process optimization.

Notebook:

<https://www.kaggle.com/code/cs23b1023akshay/wine-quality-prediction>